

NovaSMS

Sujet, Problématique, But & Objectifs — Architecture

Contexte du stage · Problème adressé · Solution · But général · Objectifs spécifiques · Méthodologie · Architecture globale · Stack technique

Stage Sankofa Lab · Romuald KO · Document régénéré depuis le code réel du dépôt
Plateforme SaaS B2B Messagerie Multi-Canal — NestJS · PostgreSQL · Redis · React 19

Table des matières

1	Contexte du stage et terrain
2	Problématique — les difficultés des PME africaines
3	Solution proposée — vision NovaSMS
4	But général du projet
5	Objectifs spécifiques
6	Périmètre fonctionnel du projet
7	Méthodologie de travail — Agile / Sprints
8	Architecture globale du système
9	Architecture logicielle — Monorepo
10	Architecture multi-tenant
11	Stack technique — Backend
12	Stack technique — Frontend
13	Sécurité — vue d'ensemble
14	Synthèse — pourquoi ces choix techniques

1.1 Cadre du stage

Ce projet a été réalisé dans le cadre d'un stage de fin d'études chez Sankofa Lab. L'objectif confié était de concevoir et développer, de bout en bout, une plateforme SaaS B2B de communication marketing multicanale destinée aux PME et entreprises d'Afrique de l'Ouest. L'historique Git du dépôt trace le développement du 23 avril 2026 (commit d'initialisation du monorepo) au 15 juillet 2026 (dernier commit à date de rédaction de ce document), soit 81 commits répartis sur environ 12 semaines de travail effectif.

1.2 Le marché visé

L'Afrique de l'Ouest, et la Côte d'Ivoire en particulier, présente un contexte où le taux de pénétration du téléphone mobile dépasse très largement celui de l'accès à Internet fixe ou à l'email professionnel. Le SMS reste, dans ce contexte, le canal de communication le plus universel : il fonctionne sur tout type de terminal, ne nécessite pas de connexion data, et affiche des taux d'ouverture nettement supérieurs à l'email. Les entreprises locales (commerces, PME de services, associations, structures de santé) ont donc un besoin réel d'outils de communication client adaptés à ce canal, mais aussi à l'email et — de façon croissante — à WhatsApp.

1.3 Constat de départ

- **Absence d'outil professionnel** ! l'envoi de SMS/emails aux clients se fait souvent manuellement, sans traçabilité
- **Pas de suivi** ! aucune visibilité sur les taux d'ouverture, de clic, ou la validité des numéros
- **Contacts fragmentés** ! bases clients dispersées dans des fichiers Excel, sans segmentation exploitable
- **Paiement local absent chez les solutions étrangères** ! les plateformes internationales (Twilio, Mailchimp) ne supportent pas nativement le Mobile Money (Wave, Orange Money, MTN MoMo, Moov Money)
- **Pas d'automatisation** ! chaque envoi nécessite une action manuelle, aucune relance ou campagne récurrente programmée
- **Multi-utilisateur limité** ! pas de gestion fine des rôles pour déléguer l'envoi à une équipe

2.1 Formulation de la problématique

? Problématique retenue

Comment concevoir une plateforme SaaS de messagerie multicanale (SMS, Email, WhatsApp) qui soit à la fois techniquement robuste (fiable, sécurisée, capable de traiter des volumes d'envoi importants), adaptée aux moyens de paiement locaux (Mobile Money + carte bancaire), et exploitable par des équipes non techniques au sein d'une PME africaine ?

2.2 Sous-problèmes techniques identifiés

- **Isolation des données** !' comment garantir qu'un compte client (tenant) ne puisse jamais accéder aux données d'un autre compte, sur une base de données mutualisée
- **Volumétrie** !' comment importer et traiter plusieurs dizaines de milliers de contacts, ou envoyer une campagne à une large audience, sans bloquer le serveur ni dégrader l'expérience
- **Fiabilité des envois** !' comment gérer les échecs (temporaires vs définitifs) de fournisseurs externes (SMS, Email) sans perdre de messages ni les envoyer en double
- **Paiement hybride** !' comment intégrer à la fois des opérateurs Mobile Money ouest-africains et un fournisseur de paiement carte (Stripe) dans un même flux de recharge de crédit
- **Sécurité des comptes** !' comment protéger l'accès à des données clients sensibles (authentification forte, 2FA, traçabilité des actions)

3.1 Présentation de la solution

NovaSMS est une plateforme SaaS B2B qui centralise, pour un compte entreprise (« Account »), la gestion de sa base de contacts, la création et l'envoi de campagnes multicanales (SMS, Email — WhatsApp partiellement intégré), l'automatisation de scénarios marketing déclenchés par des événements, le suivi analytique des performances (ouvertures, clics, désabonnements), et la gestion de son solde de crédits rechargeable par Mobile Money ou carte bancaire.

3.2 Principe de fonctionnement

- Un compte (Account) s'inscrit, vérifie son email, complète un onboarding, et peut inviter des collaborateurs (User) avec un rôle (Admin / Editor / Analyst)
- Le compte importe ou saisit ses contacts, éventuellement organisés en segments dynamiques (critères sur tags, pays, engagement...)
- Le compte crée une campagne (SMS et/ou Email), la cible sur un segment, la planifie ou l'envoie immédiatement — avec support de tests A/B
- Le compte peut définir des automatisations qui se déclenchent automatiquement sur un événement (contact ajouté, tag posé, ouverture, clic, anniversaire...)
- Chaque envoi consomme des crédits déduits du solde du compte ; le solde se recharge via Mobile Money ou carte bancaire
- Le compte peut suivre les performances de ses campagnes (tableaux de bord, heatmaps d'engagement) et exporter ses données

4 — But général du projet

%Æ But général

Concevoir, développer et livrer une plateforme SaaS B2B de messagerie multicanale complète — de l'authentification au paiement en passant par l'envoi de campagnes et l'automatisation — techniquement production-ready (architecture multi-tenant, files d'attente asynchrones, sécurité applicative), adaptée aux usages et moyens de paiement du marché ouest-africain, et validée par une suite de tests automatisés couvrant les principaux flux métier.

4.1 Ce que ce but implique concrètement

Ce but ne se limite pas à un prototype d'interface : il suppose de traiter, dès la conception, les problématiques propres à un produit multi-clients (isolation des données par compte), à fort volume (files d'attente BullMQ, traitement par lots), et à intégration de fournisseurs externes réels (SMS, Email, paiement) via un pattern d'abstraction permettant de basculer entre simulation et production sans changer le code métier.

5 — Objectifs spécifiques

5.1 Objectifs fonctionnels

- **OF-1** ! Permettre l'inscription, l'authentification sécurisée (JWT + 2FA TOTP/SMS) et la gestion d'équipe multi-rôle d'un compte entreprise
- **OF-2** ! Permettre la gestion complète d'une base de contacts : CRUD, import CSV/Excel massif, segmentation dynamique, conformité RGPD (opt-out, anonymisation)
- **OF-3** ! Permettre la création, planification, envoi et suivi de campagnes SMS et Email, avec test A/B et personnalisation de contenu
- **OF-4** ! Permettre la définition de workflows d'automatisation déclenchés par des événements métier
- **OF-5** ! Permettre la recharge du solde de crédits via Mobile Money (Wave, Orange Money, MTN MoMo, Moov) ou carte bancaire (Stripe)
- **OF-6** ! Fournir un tableau de bord analytique (taux d'ouverture, de clic, heatmaps, évolution) par campagne et par compte
- **OF-7** ! Exposer une API publique versionnée pour permettre l'intégration de NovaSMS dans les systèmes tiers des clients

5.2 Objectifs techniques / non-fonctionnels

- **OT-1** ! Isolation stricte des données entre comptes (multi-tenant) au niveau base de données et applicatif
- **OT-2** ! Traitement asynchrone des opérations lourdes (import de contacts, envoi de campagne, recalcul de segment) via des files d'attente Redis/BullMQ, pour ne jamais bloquer une requête HTTP
- **OT-3** ! Abstraction des fournisseurs externes (SMS, Email, WhatsApp, paiement, stockage) derrière un pattern Factory, avec bascule simulation !" production pilotée uniquement par variables d'environnement
- **OT-4** ! Sécurité applicative : hachage des mots de passe, blacklist JWT, rate-limiting, validation stricte des entrées, en-têtes de sécurité HTTP, vérification de signature des webhooks entrants
- **OT-5** ! Couverture par tests automatisés (unitaires backend/frontend et bout-en-bout) des flux métier critiques, intégrée à une chaîne d'intégration continue (CI)

6.1 Dans le périmètre (implémenté et fonctionnel)

- Authentification complète (inscription, connexion, 2FA, mot de passe oublié, invitations d'équipe, RBAC)
- Gestion de contacts (CRUD, import CSV/Excel, segments dynamiques, RGPD)
- Campagnes SMS et Email (wizard, éditeurs de contenu, A/B testing, planification, tracking ouverture/clic)
- Automatisations déclenchées par événements (contact ajouté, tag, ouverture, clic, anniversaire, segment)
- Paiement Mobile Money (simulation + intégration NovaSend) et carte bancaire (Stripe)
- Analytics (vue d'ensemble, rapport par campagne, heatmaps d'engagement et de clic)
- API publique v1 avec clés API et permissions granulaires
- Journal d'audit, webhooks entrants (statuts de livraison) et sortants (abonnements clients)

6.2 Hors périmètre ou partiellement implémenté (voir Doc 4 — Résultats et limites)

- **Campagnes WhatsApp** !' le provider WhatsApp (Twilio) existe côté backend, mais le canal n'est pas proposé dans le wizard de création de campagne du frontend (seulement dans les automatisations)
- **Notifications push** !' un module de providers Push (Firebase Cloud Messaging) est implémenté et testé côté backend, mais n'est raccordé à aucun module applicatif ni exposé au frontend
- **Design system frontend** !' il n'existe pas de bibliothèque de composants UI réutilisables centralisée ; chaque page réimplémente ses propres éléments visuels

7.1 Organisation en sprints

Le développement a suivi une organisation itérative proche d'une méthodologie Agile/Scrum, avec un découpage fonctionnel par domaine métier plutôt qu'une planification à intervalles fixes stricts. L'analyse rétrospective de l'historique Git permet de reconstituer les grandes phases suivantes :

Phase	Période observée (Git)	Contenu principal
Setup & socle	23 avr. 2026	Monorepo, ESLint/Prettier/Husky, Docker Compose (PostgreSQL + Redis)
Sprint Auth	Fin avr. – mi-mai 2026	Authentification, 2FA, onboarding, gestion de compte
Sprint Contacts	Mi-mai – fin mai 2026	CRUD contacts, import CSV/Excel, segments, RGPD
Sprint Campagnes	Fin mai – mi-juin 2026	Wizard de campagne, éditeurs SMS/Email, A/B testing, tracking
Sprint Automatisations	Mi-juin – début juil. 2026	Workflows, BullMQ, écoute d'événements, tests e2e
Stabilisation & finitions	Mi-juil. 2026	Validation téléphone mondiale, UX contacts/ automatisations, correctifs

Cette reconstitution est basée sur la fréquence et le contenu des commits (voir Doc 4 pour le détail semaine par semaine) ; elle sert de repère pour situer chaque module dans le temps plutôt qu'un planning figé a priori.

8.1 Vue d'ensemble

NovaSMS suit une architecture client-serveur classique à trois niveaux, complétée par une couche de traitement asynchrone dédiée aux opérations lourdes ou différées. Le frontend React communique avec une API REST NestJS via HTTPS/JSON ; l'API s'appuie sur PostgreSQL comme source de vérité (via Prisma ORM) et sur Redis à la fois comme cache et comme backend de files d'attente (BullMQ) pour les traitements asynchrones.

COUCHE CLIENTE

React 19 + Vite (SPA)
Zustand (état global persistant)
React Router 7 (routing)
Axios (HTTP + intercepteurs)

COUCHE API

NestJS 11 (REST, préfixe /api)
Guards JWT / Rôles / Clé API
ValidationPipe global
Swagger OpenAPI (/api/docs)

DONNÉES

PostgreSQL 15 (Prisma ORM)
26 modèles, 38 migrations
Isolation par accountId (tenant)

ASYNCHRONE

Redis 7 (cache + BullMQ)
5 files : dispatch, schedule,
import, automation, segment

8.2 Services externes intégrés

- **Email** !' Resend (principal) avec failover Brevo, fallback SMTP (nodemailer) en développement
- **SMS** !' Africa's Talking / Twilio / NovaSend selon configuration, via une factory avec failover automatique
- **WhatsApp** !' API Business Twilio
- **Paieement carte** !' Stripe (3D Secure)
- **Paieement Mobile Money** !' NovaSend (agrégateur simulant Wave, Orange Money, MTN MoMo, Moov Money)
- **Stockage fichiers** !' AWS S3 / MinIO (images de campagne), avec fallback stockage local en développement

9.1 Organisation du dépôt

Le projet est structuré en monorepo npm workspaces, avec deux applications indépendantes partageant l'outillage (lint, hooks Git, CI) mais sans package de types partagés généré automatiquement entre le backend et le frontend — les types d'API sont dupliqués manuellement côté frontend.

Dossier	Contenu
apps/backend	API NestJS — ~20 modules métier, Prisma, tests Jest
apps/frontend	SPA React — pages, features, composants, tests Vitest/Playwright
.github/workflows	Pipeline CI (lint, typecheck, tests backend, build)
.husky	Hooks Git (lint-staged sur commit)

9.2 Pourquoi un monorepo

Le choix du monorepo permet de garder synchronisés les contrats d'API entre backend et frontend au fil du développement rapide en itérations courtes, de mutualiser la configuration ESLint/Prettier/Husky, et de disposer d'une CI unique qui valide les deux applications à chaque push. La contrepartie observée dans le code (voir Doc 3) est l'absence de génération automatique de types partagés, qui a permis certaines divergences de contrat entre les deux applications.

10.1 Principe d'isolation

Chaque client de la plateforme correspond à une ligne de la table Account, qui constitue la racine du modèle multi-tenant. Toutes les entités métier (Contact, Campaign, Template, Segment, Transaction, ApiKey, AuditLog...) portent une clé étrangère accountId avec suppression en cascade, garantissant qu'aucune donnée ne peut exister sans rattachement à un compte, et que la suppression d'un compte purge l'intégralité de ses données.

10.2 Mise en œuvre applicative

- **TenantInterceptor (intercepteur global NestJS)** !' extrait l'accountId du JWT décodé et l'injecte dans la requête (req.accountId) pour tous les contrôleurs
- **RolesGuard + décorateur @RequireRoles** !' restreint certaines routes à des rôles précis (Admin / Editor / Analyst) au sein d'un même compte
- **ApiKeyGuard** !' pour l'API publique, l'identification du tenant se fait par clé API (hash SHA-256 stocké, jamais la clé en clair) plutôt que par JWT

Domaine	Technologie	Rôle dans NovaSMS
Framework	NestJS 11 (TypeScript 5.7)	Structure modulaire, injection de dépendances, décorateurs
ORM / BDD	Prisma 6 + PostgreSQL 15	Accès typé aux données, migrations versionnées (38 migrations)
Cache / Files	ioredis + BullMQ 5	Cache segments/comptages, files d'attente asynchrones
Auth	@nestjs/jwt, passport-jwt, bcryptjs, sneakeasv	JWT access/refresh, hash mdp, TOTP 2FA
Validation	class-validator + zod (coexistant)	Validation des DTOs entrants
Sécurité HTTP	helmet, @nestjs/throttler	En-têtes sécurisés, rate-limiting
Paieement	stripe (SDK officiel)	Recharge carte bancaire + 3D Secure
Téléphonie	libphonenumber-js	Validation/normalisation E.164, 245 pays
Fichiers	multer, exceljs, @aws-sdk/client-s3, pdfkit	Upload, import Excel, stockage S3, génération de reçus PDF
Documentation API	@nestjs/swagger	Documentation OpenAPI interactive (/api/docs)
Tests	Jest 29 + Supertest	Tests unitaires (67 fichiers) et e2e (6 fichiers)

Domaine	Technologie	Rôle dans NovaSMS
Framework	React 19.2 + Vite 8 (TypeScript ~6.0)	SPA avec code-splitting par route (React.lazy)
Routing	react-router-dom 7 (BrowserRouter)	Navigation, routes protégées et guardées par rôle
État global	Zustand 5 (+ middleware persist)	authStore, uiStore, campaign.store (persistés en localStorage)
Style	Tailwind CSS 3.4 + CSS custom (index.css)	Mise en page utilitaire + classes maison (.card, .kpi...)
Formulaires	react-hook-form 7 + zod 4 (usage partiel)	Validation du formulaire d'inscription
HTTP	axios 1.15 (intercepteurs)	Injection JWT, refresh automatique sur 401, toasts d'erreur
Feedback UI	sonner (toasts)	Retours utilisateur succès/erreur
Divers	papaparse, qrcode, framer-motion, driver.js	Parsing CSV, QR code 2FA, animations, onboarding guidé
Tests	Vitest 4 (unitaires) + Playwright 1.59 (e2e)	2 fichiers unitaires, 10 fichiers e2e (~45 scénarios)

Remarque méthodologique : cette table reflète les dépendances réellement importées et utilisées dans le code (vérifié par recherche croisée), et non la simple liste du package.json — certaines dépendances déclarées (recharts, @dnd-kit, class-variance-authority...) ne sont jamais utilisées dans src/ ; ce point est détaillé et assumé comme dette technique dans le Doc 4.

13.1 Mesures en place

- Mots de passe hachés avec bcryptjs (12 rounds), jamais stockés ni journalisés en clair
- Authentification par JWT à double secret (access / refresh), révocation immédiate possible via une blacklist Redis au logout
- Deuxième facteur d'authentification (2FA) au choix : application TOTP (speakeasy) ou code par SMS, avec codes de secours
- Verrouillage de compte après 5 tentatives de connexion échouées (15 minutes)
- Clés API jamais stockées en clair (hash SHA-256), permissions granulaires par clé, rate-limiting dédié
- Vérification de signature HMAC-SHA256 sur les webhooks entrants (Resend, Africa's Talking, Twilio, Stripe)
- En-têtes de sécurité HTTP (helmet), CORS restreint par liste blanche, limite de taille de payload (10 Mo)
- Journal d'audit (AuditLog) alimenté par les modules sensibles (auth, contacts, campagnes, tracking)
- Anonymisation RGPD automatique (cron quotidien) des contacts désabonnés depuis plus de 30 jours

Q — Pourquoi deux librairies de validation (class-validator ET zod) coexistent-elles dans le backend ?

Ce n'est pas un choix architectural délibéré mais une évolution progressive du code au fil des sprints : les premiers modules (campagnes, templates, automatisations) ont été écrits avec class-validator, standard historique de NestJS ; les modules développés plus tard (auth, segments) utilisent zod, plus léger et davantage utilisé côté frontend, ce qui a facilité le partage mental des schémas entre les deux équipes de développement. Cette double approche est documentée comme point de dette technique à trancher dans une prochaine itération (voir Doc 4).

% Pourquoi NestJS plutôt qu'Express nu ou Fastify seul

NestJS impose une architecture modulaire (modules / contrôleurs / services / DTOs) proche de Spring ou Angular côté serveur, avec injection de dépendances native. Pour un projet à 20 modules métier développé en solo sur un temps court, cette structure impose une discipline qui limite la dette d'architecture, et fournit nativement l'intégration avec Passport (JWT), BullMQ, Swagger et le système de Guards/Interceptors utilisé pour le multi-tenant.

% Pourquoi Prisma plutôt qu'un ORM comme TypeORM ou Sequelize

Prisma génère un client TypeScript entièrement typé à partir du schéma, ce qui supprime une classe entière d'erreurs de frappe sur les noms de champs/relations et permet l'autocomplétion sur les 26 modèles du projet. Son système de migrations versionnées (38 migrations dans le dépôt) donne un historique clair de l'évolution du schéma de données au fil des sprints.

% Pourquoi Redis + BullMQ pour l'asynchrone

Les opérations d'import de contacts (jusqu'à plusieurs dizaines de milliers de lignes) et d'envoi de campagnes ne peuvent pas s'exécuter dans le cycle de vie d'une requête HTTP sans risquer un timeout. BullMQ, adossé à Redis, permet de découper ces traitements en jobs retryables avec backoff exponentiel, tout en gardant une API simple d'intégration avec NestJS via `@nestjs/bullmq`.

% Pourquoi React + Zustand plutôt qu'un framework avec state management intégré (Redux, Recoil)

Zustand offre une API minimale (pas de reducers/actions boilerplate) suffisante pour les trois besoins réels de l'application (session auth, préférences UI, brouillon de campagne), avec une persistance localStorage intégrée. Ce choix a permis d'aller vite sur un projet à un seul développeur frontend, au prix d'une absence de couche de cache serveur (pas de React Query), compensée manuellement par des `useEffect` — un point discuté comme piste d'amélioration dans le Doc 4.

% Pourquoi séparer Mobile Money et Stripe plutôt qu'un fournisseur de paiement unique

Aucun fournisseur de paiement international ne couvre nativement les opérateurs Mobile Money ouest-africains (Wave, Orange Money, MTN MoMo, Moov Money), pourtant majoritaires dans les usages de paiement des PME ciblées par NovaSMS. Le projet implémente donc deux familles de providers (MobileMoneyTransaction vs Transaction/Stripe) derrière des factories dédiées, avec un mode simulation permettant de développer et tester sans dépendre de comptes marchands réels.